

BOB ON THE FLOOR

Bob on the Floor

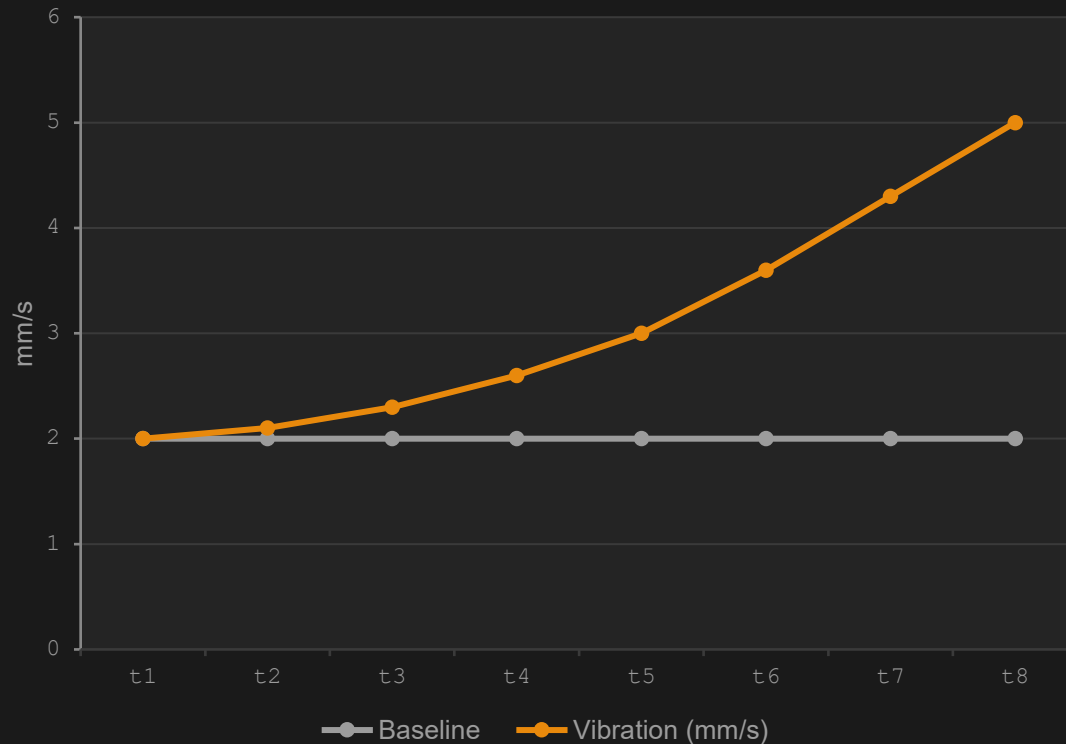
LLM-Orchestrated Industrial Fault Response, MCP-Native

Conveyor motor anomaly -> procedure retrieval -> ladder-logic patch -> human approval

Five devices. Eight MCP tools. Zero ML dependencies.

The 2 AM Problem

Sensor drift above established baseline



- ▶ A conveyor motor's vibration drifts above its established baseline
- ▶ No automated system connects the sensor anomaly to the maintenance procedure
- ▶ An engineer must: recognise the fault pattern, search the manual, write a ladder-logic patch, and get approval — manually, under time pressure
- ▶ Missed or delayed response: bearing failure, unplanned downtime, safety risk

One Pipeline. Five Stages. Fully Composable.

Bob on the Floor is a five-device fault-response pipeline exposed as discrete MCP tools. Any MCP-capable LLM orchestrator — IBM Bob, Claude, or any agent framework — can drive each stage independently or run the whole chain in a single call.

```
Telemetry  ->  Anomaly  ->  Procedure  ->  Ladder  ->  Approval
Schema 1      Schema 2      Schema 3      Schema 4      Schema 5
```

```
MCP tools: listen_telemetry -> detect_anomaly -> retrieve_procedure -> compile_ladder ->
approve_change
```

Every stage is its own MCP tool — the orchestrator inspects each output before deciding whether to call the next.

Five Devices, Five Schemas, Eight Tools



Every inter-stage boundary is enforced at runtime by `jsonschema`. Schema violations are hard MCP errors — not silent downstream `KeyErrors`.

No device imports another device's internals. If output violates the contract, the pipeline stops at that boundary with a clear error — the chain is transparent and inspectable end to end.

Ears: Telemetry Ingestion | Instinct: Anomaly Detection

EARS

MCP tool: `listen_telemetry`

Returns Schema 1: `device_id`, `timestamp`,
`readings.vibration_mm_s`, `readings.temperature_c`,
`readings.rpm`

Pluggable source via `EARS_SOURCE`

- `fixture` — cycles `telemetry_stream.jsonl` (default, no hardware)
- `modbus` — live Modbus RTU via `EarsModbusRtuAdapter`
- `mqtt` — stub (contribution point)

INSTINCT

MCP tool: `detect_anomaly` — Stateful, per-device anomaly detector. Zero ML.

- Learn baseline: 100-tick rolling window (frozen once full)
- Dual-threshold flag: >12% deviation and z-score >3.0
- Confirm on 2 consecutive flagged ticks
- Re-arm after 3 consecutive settled ticks (hysteresis)
- Re-emit if severity climbs >0.10, 10-tick cooldown

`vibration + temperature` → `bearing_wear`

`vibration only` → `vibration_drift`

`temperature only` → `thermal_creep`

`rpm only` → `other`

Memory: Procedure Retrieval

MCP tool: `retrieve_procedure`

Accepts Schema 2 (anomaly flag). Uses the description field as a free-text query. Returns Schema 3: matched procedure, source document, page number, excerpt, suggested_fix_nl, confidence score.

HOW IT WORKS

- ▶ BM25Okapi index over corpus.json — 10 procedure entries from conveyor maintenance manuals, inspection tickets, and SOPs
- ▶ No embedding model. No network calls at query time. Fully deterministic.
- ▶ Hot-reload without restart: call `reload_corpus` MCP tool after uploading a new manual

EXAMPLE MATCH

```
Anomaly description:
"vibration rising 28% above baseline on conveyor_motor_2"

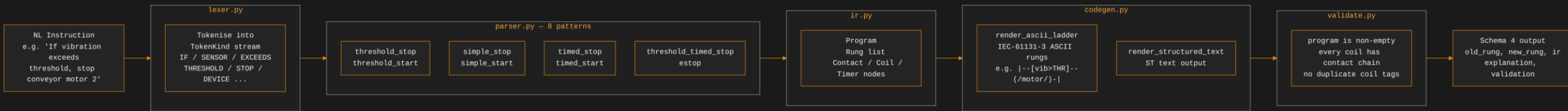
PROC-VIB-014 "Conveyor Motor Maintenance Manual" p.14
confidence: 0.91

suggested_fix_nl: "If vibration exceeds threshold, stop
conveyor motor 2."
```

Hands: NL → Ladder Logic Compiler

MCP tool: `compile_ladder`

Accepts Schema 3. Compiles suggested_fix_nl into a Schema 4 ladder-logic diff. No LLM — a deterministic lexer/parser/IR/codegen pipeline.



EXAMPLE

IN: "If vibration exceeds threshold, stop conveyor motor 2."

OUT: |--[vib>THR]-- (/conveyor_motor_2/)-|

validation: **PASS**

[PASS] program is non-empty

[PASS] every coil has contact chain

Approval: Human-in-the-Loop, Four Modes

MCP tool: `approve_change` — mode controlled by `APPROVAL_MODE` env var

Mode	Behaviour
<code>interactive</code>	Rich terminal UI — blocks for operator input
<code>auto_approve</code>	Returns approved immediately — CI / known-safe fixtures
<code>auto_reject</code>	Returns rejected immediately — pipeline validation tests
<code>webhook</code>	POSTs Schema 4 to <code>APPROVAL_WEBHOOK_URL</code> , polls every 5s, 300s timeout

mocked interactive terminal ui

APPROVAL REQUIRED — device: conveyor_motor_2

device_id: conveyor_motor_2 dialect: IEC-61131-3

Explanation: stops conveyor motor 2 when vibration exceeds the configured threshold.

✓ program non-empty ✓ coil-contact chain

PASS

OLD RUNG
|--[vib]--()--|

NEW RUNG
|--[vib>THR]--(/motor_2/)-|

y approve n reject skip defer >

Six Technical Differentiators

1**MCP-native pipeline composition**

Five independent MCP tools. Any orchestrator can call each stage, inspect the output, and decide whether to proceed.

2**Deterministic anomaly detection, no ML**

Dual-threshold (12% + $z=3.0$), 2-tick confirmation, 3-tick hysteresis. Fully auditable — parameters live in code.

3**NL → Ladder Logic compiler**

Real lexer/parser/IR/codegen for 8 patterns. Produces valid IEC-61131-3 ASCII rungs. No LLM in the compile path.

4**Schema-enforced inter-stage contracts**

jsonschema validates every MCP tool return. A violation is a hard error at the boundary, not a `KeyError` downstream.

5**Fully offline procedure retrieval**

BM25 over a local corpus. No embedding model, no network call at query time. Hot-reloadable via `reload_corpus`.

6**Zero shared state via FastMCP lifespan**

Every component lives in one `PipelineContext`, injected via lifespan. No singletons — full parallel test isolation.

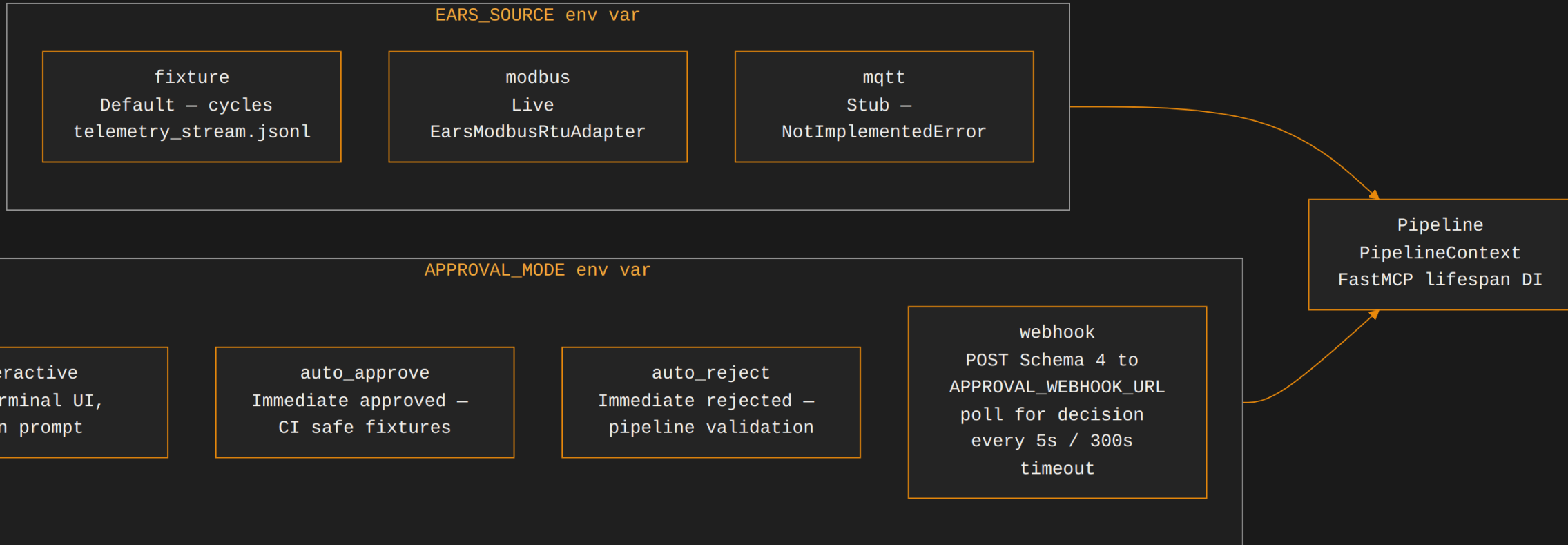
```
$ live demo
```

```
run_pipeline_tick           # full pipeline in one call
```

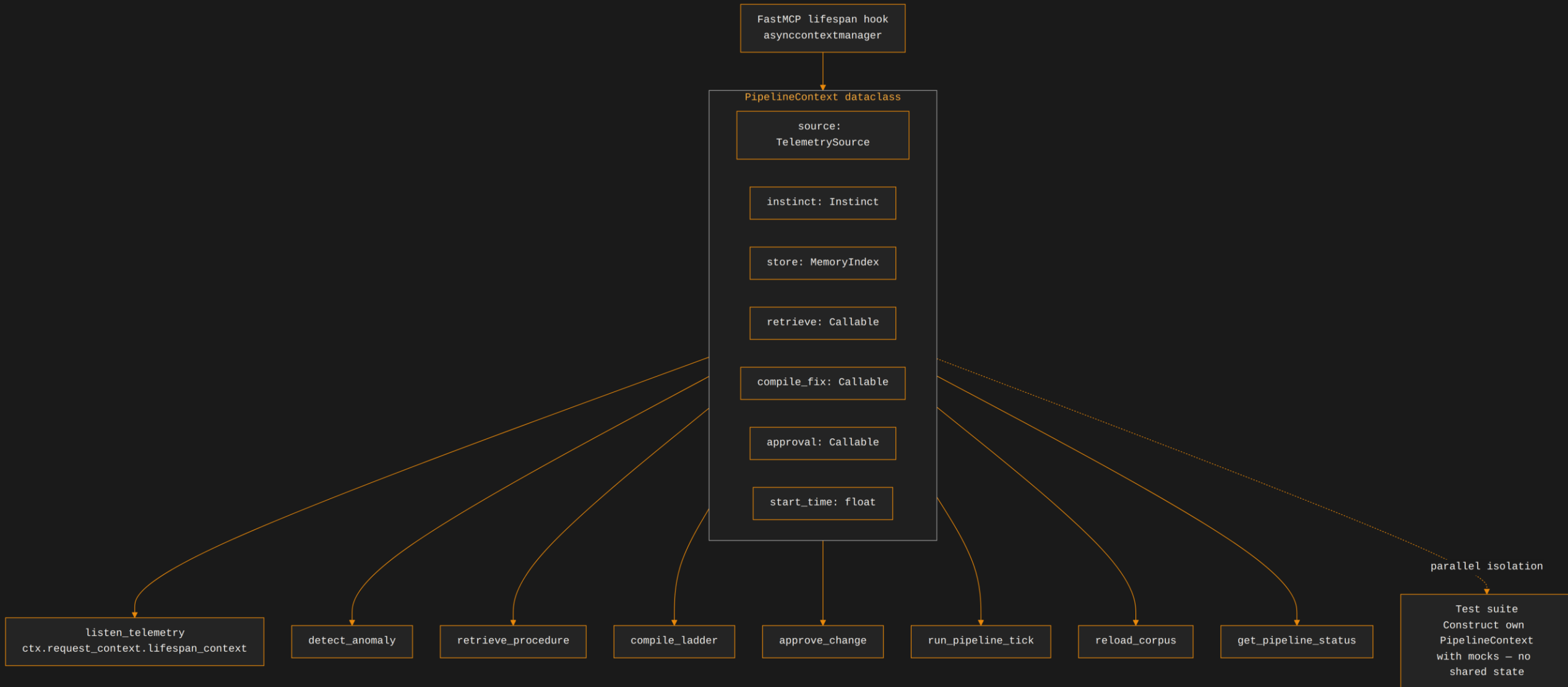
```
listen_telemetry -> detect_anomaly -> retrieve_procedure  
-> compile_ladder -> approve_change      # step by step
```

- ▶ interactive approval terminal UI
- ▶ schema violation: hard error at the boundary

Pluggability Points



Dependency Injection Architecture



Tech Stack

Component	Technology
MCP server	mcp (FastMCP, stdio transport)
Anomaly detector	Pure Python – math, collections.deque
Procedure retrieval	rank_bm25, jsonschema
NL compiler	Pure Python – lexer/parser/IR/codegen
Approval UI	rich
Telemetry source	pymodbus (Modbus RTU), .jsonl fixture
Schema validation	jsonschema (all 5 schemas, every tool return)
Configuration	Environment variables or YAML (PyYAML)

Total runtime dependencies: mcp, jsonschema, rank_bm25, rich, pymodbus, PyYAML

Zero ML framework dependencies.

Designed to Be Extended

Add a new telemetry source

Subclass `TelemetrySource`, implement `next_tick()`, set `EARS_SOURCE`. Example: OPC-UA adapter, MQTT consumer.

```
class OpcUaSource(
    TelemetrySource):
    def next_tick(self):
        ...
    EARS_SOURCE=opcua
```

Extend the procedure corpus

Add entries to `corpus.json` (or point `CORPUS_PATH` at a new file), call `reload_corpus`. No restart, no reindex command.

```
$ edit corpus.json
$ mcp call \
    reload_corpus
# BM25 index rebuilds
# in-process
```

Add an NL pattern to Hands

Add a pattern function to `parser.py` following the existing 8-pattern structure. First match wins.

```
def threshold_stop(
    tokens): ...
PATTERNS = [
    ..., new_pattern
]
```

Because every stage is isolated by schema contracts, extending one device never requires touching another — the schema is the only interface.

Get It Running in Two Steps

STEP 1

```
$ pip install -r requirements.txt
```

STEP 2 — add to mcp.json

```
{
  "mcpServers": {
    "bob-on-the-floor": {
      "command": "python",
      "args": ["/absolute/path/to/mcp_server.py"]
    }
  }
}
```

That's it.

Bob spawns the server on demand.

AVAILABLE TOOLS IMMEDIATELY

listen_telemetry	retrieve_procedure	run_pipeline_tick
detect_anomaly	compile_ladder	reload_corpus
approve_change		get_pipeline_status

Repository: (add your GitHub URL here)